開発者が最初に読むマニュアル

関数を作る → 手元で確かめる → Google Cloudに配置する → 運用する

Google Cloud Run functions

サーバーの構築・管理を意識せず、HTTPリクエストやイベントを受け取って処理を実行するための仕組みです。現在の関数はCloud Runのサービスとして動きます。

この1冊でできるようになること

Pythonの小さなHTTP関数を作り、WindowsのPowerShellでローカル実行し、認証が必要な開発用サービスへデプロイして結果とログを確認します。

読む順番	到達点
02 全体像と設計	何をきっかけに、どの処理を動かすか決める
03 環境と権限	プロジェクト・API・4つの主体を把握する
04–05 実装と確認	main.pyを作り、正常系と異常系を確かめる
06–07 配置と検証	非公開でデプロイし、IDトークンで呼ぶ
08–09 改修と業務応用	変更を反映し、障害・イベント処理に備える
10–14 コスト最適化	成功した業務1件の費用を計測して改善する
15–16 公式情報	各章の根拠と詳細を調べる

対象と前提

基本的なPythonとHTTPの知識がある開発者向けです。実習ではPython 3.13、VS Code、Google Cloud CLIを使います。Python 3.13は学習用の選択で、最新版という意味ではありません。

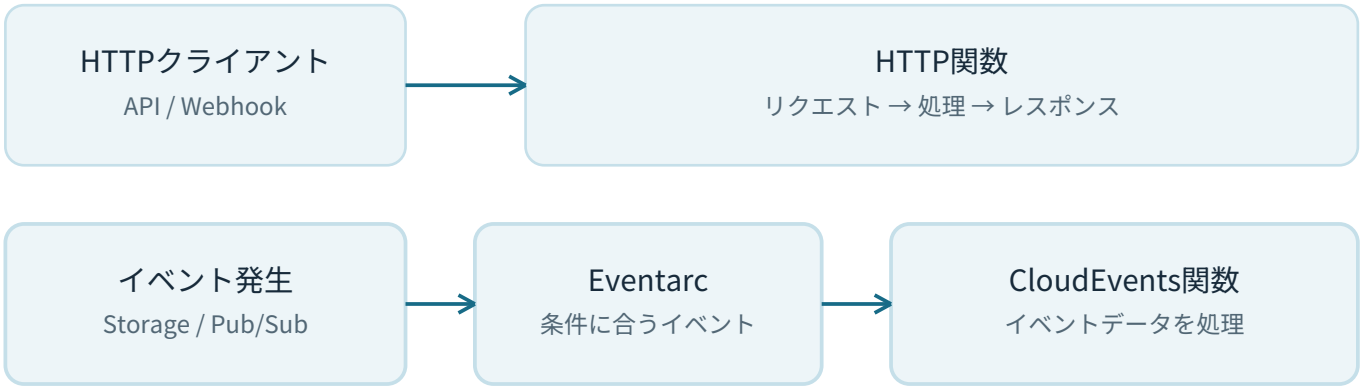
本書はCloud Run Admin APIとgcloud runを使う新規開発の流れです。既存のgcloud functions deployや第1世代の手順とは設定名が異なるため、混在させないようにします。

参照：[01], [02], [04], [15] / URL一覧は15–16ページ

01 まず全体像をつかむ

実装前に「入力・処理・出力・起動方法」を決めます。

起動方法は大きく2種類



本書は上段のHTTP関数で基本を習得します。下段はファイルのアップロードやメッセージ受信をきっかけに処理する構成です。イベントトリガーの設定は、HTTP関数のデプロイとは別に行います。

実装前の確認	本書の実習での決定
起動と入力	GET / クエリパラメーター name
処理と出力	あいさつ文字列を生成し、JSONを返す
異常時	GET以外は405、nameが長過ぎる場合は400
呼び出し元	run.invokerを持つ開発者だけ
配置先	開発プロジェクト / 東京リージョン

ソースを配置すると何が起きるか



ビルドされたイメージはArtifact Registryに保存されます。このソースデプロイでは、開発者のPCにDockerを用意する必要はありません。

参照：[02], [03], [09] / URL一覧は15–16ページ

02 環境と権限を準備する

会社の既存プロジェクトを使う場合は、管理者と役割・配置先をそろえます。

Google Cloudの開発用プロジェクトと課金の有効化を確認します。PCにはPython 3.13、VS Code、Google Cloud CLIを用意します。以降のコマンドはPowerShell用です。PROJECT_IDはプロジェクト名ではなくIDへ置き換えます。

```
gcloud init
gcloud config set project PROJECT_ID
gcloud auth list
gcloud config get-value project
gcloud services enable artifactregistry.googleapis.com `
  cloudbuild.googleapis.com run.googleapis.com `
  logging.googleapis.com
```

権限は「4つの主体」に分けて理解する

主体	役割・確認する権限
開発者（デプロイ）	run.sourceDeveloper / serviceusage.serviceUsageConsumer 実行用SAに対するiam.serviceAccountUser
ビルド用SA	プロジェクトのrun.builder 既定か専用か、実際のビルド設定で確認する
呼び出す人・サービス	対象サービスのrun.invoker デプロイ権限とは別に考える
実行用SA	関数からアクセスするAPI・データへの権限 本書のあいさつ関数は業務データ権限不要

SAはサービスアカウントです。APIの有効化にはserviceusage.services.enableが必要です。実行用SAの作成、呼び出し権限の付与、ログ閲覧にもそれぞれの権限が必要です。権限設定担当者に依頼し、開発者へ一律にOwnerを付ける運用は避けます。

ローカルのGoogle API認証は別の設定

本書の実習では不要です。後でPythonからBigQueryなどのGoogle APIを呼ぶ場合、ローカル用ADCを設定します。gcloudのログインと、Pythonが使う認証は同じ設定ではありません。

```
gcloud auth application-default login
```

参照：[03], [06], [08], [13] / URL一覧は15–16ページ

03 最小の関数を実装する

サービス名は hello-function、Pythonの入口関数名は hello です。

hello-functionフォルダー内	役割
main.py	HTTPの入力を受け取り、結果を返す
requirements.txt	クラウドとローカルで使う依存関係
.gcloudignore	送らないファイルを指定する

main.py

```
import logging
import functions_framework
@functions_framework.http
def hello(request):
    """Return a greeting for a validated GET request."""
    if request.method != "GET":
        return {"error": "method_not_allowed"}, 405, {
            "Allow": "GET"
        }
    name = request.args.get("name", "World").strip()
    if len(name) > 50:
        return {"error": "name_too_long"}, 400
    logging.warning("greeting_request_processed")
    return {"message": f"Hello, {name or 'World'}!"}, 200
```

requestはHTTPリクエスト、helloがエントリーポイントです。辞書はJSONレスポンスになります。サンプルのログは結果確認用にWARNINGで出しています。業務ではログレベルを定義し、正常処理はINFOなどに調整します。

requirements.txt / .gcloudignore

```
# requirements.txt
functions-framework==3.9.2
# .gcloudignore (create as a separate file)
.gcloudignore
.git/
.venv/
__pycache__/
.env
.env.*
```

ファイルはUTF-8で保存します。依存バージョンは本書の実習例で固定しています。実務では互換性・脆弱性を確認して更新し、機密ファイルを送信対象に含めないようにします。

参照：[02], [04], [05], [07] / URL一覧は15–16ページ

04 ローカルで動かして確かめる

クラウドへ送る前に、入口とレスポンスを手元で確認します。

ターミナルA：作業フォルダーで起動

```
cd hello-function
py -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\functions-framework.exe `
  --target hello --port 8080
```

仮想環境の有効化を省き、実行ファイルのパスを直接指定しています。起動したターミナルは開いたままにします。helloはmain.pyの関数名と完全一致させます。

ターミナルB：HTTPリクエストを送る

```
Invoke-RestMethod "http://localhost:8080/?name=Developer"
curl.exe -i "http://localhost:8080/"
curl.exe -i -X POST "http://localhost:8080/"
$longName = "x" * 51
curl.exe -i "http://localhost:8080/?name=$longName"
```

確認するケース	期待結果
GET /?name=Developer	200 / message = Hello, Developer!
GET /（nameなし）	200 / message = Hello, World!
POST /	405 / error = method_not_allowed
nameが51文字	400 / error = name_too_long

ローカル確認の完了条件

4つのケースで期待どおりのHTTPステータス・本文が返り、ターミナルAで処理ログが見えることを確認します。停止するときはターミナルAでCtrl+Cを押します。

この確認は手動の動作確認です。業務の計算ロジックを追加したら、HTTP処理から切り出して単体テストを作ります。認証・IAM・ネットワーク・クラウドAPI連携は配置後にも検証します。

参照：[05], [10] / URL一覧は15–16ページ

05 開発環境へデプロイする

本書では認証必須の新規サービスを作ります。コマンドは実習用です。

実行用サービスアカウントを用意

管理者が作成済みの専用アカウントを使う場合は、それを指定します。以下は作成権限を持つ設定担当者向けの例です。PROJECT_IDを実際のIDへ置き換えます。

```
gcloud iam service-accounts create hello-runtime `
  --display-name="Hello function runtime"
```

main.pyがあるフォルダーで実行

```
gcloud run deploy hello-function `
  --source . `
  --function hello `
  --base-image python313 `
  --region asia-northeast1 `
  --service-account hello-runtime@PROJECT_ID.iam.gserviceaccount.com `
  --no-allow-unauthenticated `
  --max-instances 2 `
  --timeout 60
```

指定	意味
--source . / --function hello	現在のフォルダーを送信 / 入口関数を選ぶ
--base-image python313	クラウドでもPython 3.13を使う
--region asia-northeast1	東京に配置する実習例
--service-account	関数が動作するときのGoogle Cloud上のID
認証・インスタンス数・時間	認証必須 / 最大数の設定 / 60秒のタイムアウト

最大インスタンス数は費用を抑える補助設定で、厳密な予算上限ではありません。組織のリージョン制限や設定に従い、完了時のサービスURLとリビジョンを控えます。

コンソールを使う場合はCloud Runの「関数を作成（Write a function）」からサービス名・リージョン・ランタイム・認証を設定し、ソースと入口関数を指定します。本書の実習は設定を再現しやすいCLIを中心にしています。

参照：[03], [08], [14], [15], [16] / URL一覧は15–16ページ

06 クラウドで呼び出し、ログを見る

デプロイ成功と、関数が期待どおり動くことは別々に確認します。

呼び出し権限を付ける（権限設定担当者）

```
gcloud run services add-iam-policy-binding hello-function `
  --region asia-northeast1 `
  --member="user:YOUR_EMAIL" `
  --role="roles/run.invoker"
```

YOUR_EMAILはテストする開発者のGoogleアカウントへ置き換えます。すでに必要な呼び出し権限がある場合、重ねて付与する必要はありません。

開発者：URLを取得して認証付きで呼ぶ

```
$serviceUrl = gcloud run services describe hello-function `
  --region asia-northeast1 --format="value(status.url)"
$token = gcloud auth print-identity-token
$headers = @{ Authorization = "Bearer $token" }
Invoke-RestMethod -Uri "$serviceUrl/?name=Developer" `
  -Headers $headers
```

開発用のIDトークンをAuthorizationヘッダーで送ります。期待結果は200とHello, Developer!です。トークンは共有・記録しないでください。本番のサービス間認証では、対象サービスのaudienceを指定したIDトークンを使います。

ログと配置設定を確認

Google Cloudコンソール → Cloud Run → hello-function → ログで、リクエストとgreeting_request_processedを確認します。リビジョンとセキュリティ設定も確認し、認証なしのアクセスが成功しないことを確かめます。

完了条件	確認内容
正常系・異常系	ローカルと同じ4ケースを認証付きで確認
認証・ログ	認証なしでは拒否 / 処理ログが表示される
配置先	プロジェクト・リージョン・実行用SAが予定どおり

参照：[06], [07], [12] / URL一覧は15–16ページ

07 修正・再配置・障害調査

運用中の変更は、ソース・設定・検証結果をセットで残します。

普段の開発サイクル



配置後は新しいリビジョンと動作・ログを確認します。本番への反映は、チームの受入条件を満たしてから行います。問題がある場合は、Cloud Runのリビジョン・トラフィック管理で以前のリビジョンへ戻す方法を検討します。

症状	最初に見る場所・確認事項
ビルド失敗	Cloud Buildのログ / 依存関係 / ビルド用SA
起動失敗	Cloud Runのログ / main.py / helloの綴り
401・403	IDトークン / 呼び出すID / run.invoker 組織ポリシーやIngress設定も確認
500	アプリケーションログ / 例外 / 外部APIの権限
応答が遅い・時間切れ	処理時間 / 外部API / 初回起動 / timeout

実務で先に習慣にすること

・環境変数で環境ごとの差を分け、パスワード等はSecret Managerで管理します。
・処理結果をローカルファイルに永続保存する前提にせず、必要なデータは外部へ保存します。
・HTTP応答を返した後に、必要なバックグラウンド処理を続ける設計は避けます。
・不要な依存を増やさず、ログに入力全文・個人情報・認証情報を残さないようにします。

学習後の片付け

使わない実習サービスは削除します。Artifact Registryのイメージ等は別に残る場合があります。共有リポジトリをまとめて削除せず、不要な実習リソースだけを確認して整理します。

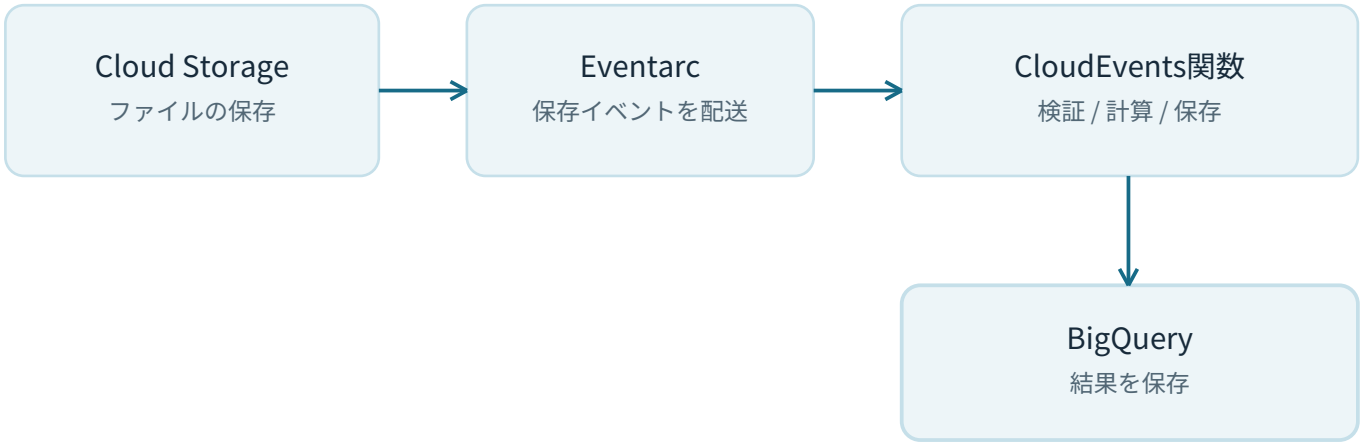
```
gcloud run services delete hello-function `
  --region asia-northeast1
```

参照：[07], [10], [11], [12], [14] / URL一覧は15–16ページ

08 業務処理へ広げる考え方

例：ファイル受信 → 計算 → BigQueryへ保存。本書では構成の理解まで扱います。

イベント処理では「重複」を前提にする



イベントの再配送や処理の再実行で、同じデータを処理する可能性があります。ファイル名と世代番号などで処理対象を識別し、同じ入力を再処理しても結果が二重登録されない設計（冪等性）にします。再試行の扱いはトリガー方式ごとに確認します。

入口・計算・保存を分ける

ファイル例（設計例）	担当する処理
main.py	HTTPまたはCloudEventを受け、入力を検証する
calculation.py	入力値から結果を計算する。単体テストの中心
repository.py	Storage読込・BigQuery書込などのI/O
tests/	計算・入力境界・再実行の挙動を確認する

共通モジュールは、送信するソースフォルダー内へ含めるか、依存パッケージとして配布します。開発PCにあるデプロイ対象外のフォルダーを、そのままクラウドからimportすることはできません。

呼び出し権限とデータ操作権限は別

Eventarc用SAには関数を呼ぶ権限を、関数の実行用SAには対象のStorageやBigQueryへの必要な権限を付けます。ローカルで動くことだけでは、クラウドで同じ権限が使えると判断できません。

業務化の前に決める項目：入力形式、計算仕様、失敗時の保存先、再試行・重複防止、監視、タイムアウト、データの配置リージョン。上記ファイル構成は本書の設計例です。

参照：[02], [08], [09], [10], [13] / URL一覧は15–16ページ

09 1件あたりの費用を理解する

最適化の目的：必要な応答速度と成功率を守り、業務処理1件の総費用を下げます。

「HTTPを1回呼ぶ」と「業務1件を完了」は区別する

1回の呼び出しを安くしても、失敗や再試行が増えると業務全体では高くなります。本書では、重複を除いた成功件数で費用を割る指標を使います。以下の指標・比較手順は本書の運用設計例です。

```
Cost per successful item = Total cost / Successful unique items
```

総費用に含める項目	確認する内容
Cloud Runの実行	割り当てCPU・メモリと課金対象時間、リクエスト数
起動・待機	初期化、終了処理、最小インスタンスの待機費用
外部サービス	BigQuery、Storage、外部API、通信、ログ等
ビルド・保管	Cloud Build、Artifact Registryを期間・件数で配賦

まずは単独実行の概算で感覚をつかむ

```
Compute cost ~= T * (V * Pc + M * Pm)
Request cost ~= R * Pr
```

T＝課金対象秒数、V＝割り当てvCPU数、M＝割り当てGiB、Pc・Pm＝各秒単価、R＝課金対象リクエスト数、Pr＝リクエスト単価です。CPUを実際に使った割合ではなく、割り当てたリソースが基準です。

上の式はリクエストベース課金・同時実行なし・起動ブースト等を省いた概算です。外部API待ちもリクエスト処理中の時間に含まれます。課金時間は100ミリ秒単位で切り上げられ、起動・終了も対象になります。同時実行ではリクエストの重なった時間を二重加算しません。

無料枠と価格の扱い

無料枠は請求先アカウント単位で集計されるため、関数ごとに「無料」と判断しません。比較時は無料枠適用前の費用と実際の請求を分けます。単価はリージョン・課金方式・通貨に応じて公式料金表から取得してください。

参照：[14], [17], [18], [25], [28] / URL一覧は15–16ページ

10 処理時間・CPU・メモリを調整する

小さい設定が常に安いとは限りません。料金と時間を掛け合わせて比べます。

最初は設定より、処理の無駄を減らす

① 必要な列・行だけ取得する → ② 同じ計算・API呼び出しを繰り返さない → ③ 読込・保存をまとめる → ④ CPU・メモリを調整する、の順で検討します。順序は本書の推奨で、処理時間の内訳を測って判断します。

処理の特徴	改善候補と注意点
CPU中心の計算	計算量を減らす。複数CPUは並列利用できる実装で比較
API・DB待ちが中心	接続を再利用、無駄な往復を削減。CPU増だけでは待ち時間は減らない
大きなファイル・データ	必要分だけ読む、分割処理する。全件をメモリに保持しない
メモリ不足・OOM	メモリを増やすか実装を改善。再試行増加も費用に含める

CPUを増やす価値は、短縮できた時間で決まる

比較例（仮定値）	単独実行のCPU・メモリ費用
A：1 CPU / 1 GiB / 4.0秒	4.0 Pc + 4.0 Pm（基準）
B：2 CPU / 1 GiB / 2.8秒	5.6 Pc + 2.8 Pm（CPU時間は増える）
C：2 CPU / 1 GiB / 1.0秒	2.0 Pc + 1.0 Pm（両方減る）

この時間は説明用の仮定で、実測値ではありません。Bは速くても割高になる可能性があり、Cなら計算資源費用は下がります。起動時間や料金丸め等も加え、同じ入力・同じ結果で比べます。

メモリとPythonの並列性

メモリは平常時の平均ではなく、最大入力・同時実行時のピークと余裕で決めます。CPUとメモリには選択可能な組み合わせがあります。単一スレッドの計算はCPUを増やしても比例して速くなるとは限らないため、実際の並列利用を確認します。

参照：[10], [19], [20], [23], [24], [25] / URL一覧は15–16ページ

11 同時実行と初回起動を最適化する

1リクエストの速さと、全体の処理効率を両方確認します。

同時実行でリソースを共有する



I/O待ちが多い処理は、待機中に別リクエストを処理できると効率が上がります。ただしconcurrencyは受け入れ上限で、設定した値だけ実際に並列処理される保証はありません。1件ずつしか来ない場合には、この効果は得られません。

調整項目	実務での判断
同時実行数	I/O中心は8などの低い値から比較。CPU・メモリ集中や共有状態がある処理は1も検討
Pythonの実行スレッド	Web実行側も合わせて調整。ソースデプロイではTHREADS=8等を検討
共有オブジェクト	クライアント再利用はライブラリのスレッド安全性を確認。ユーザー入力を共有変数へ保存しない
メモリ・接続数	同時実行数を増やした負荷でピークと外部APIの上限を確認

コールドスタート：毎回の準備を減らす

不要な依存を外し、使い回せるAPIクライアントや不変の設定をインスタンス内で再利用します。高価な初期化は必要性と測定結果で配置を決めます。インスタンスは再利用されない場合もあるため、キャッシュの存在を正しさの前提にしません。

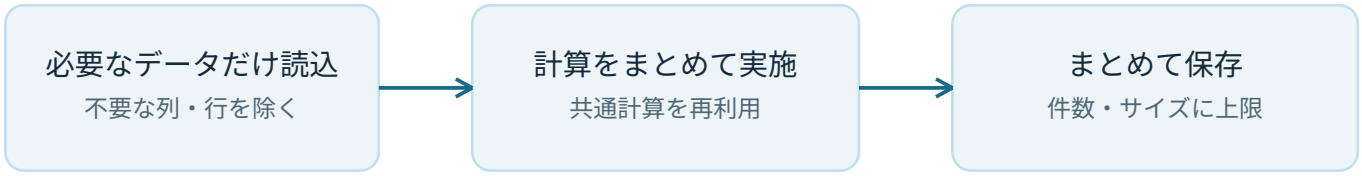
設定	費用と速度のトレードオフ
最小インスタンス0	低頻度実行の費用を抑えやすい。初回の遅延を測る
最小インスタンス1以上	初回遅延を抑える候補。待機費用が増え、新規起動は残る
Startup CPU boost	起動を速くする候補。起動後10秒もCPU増量対象で、増量分の費用も比較

参照：[10], [19], [21], [22] / URL一覧は15–16ページ

12 外部費用と再実行を減らす

Cloud Runの数秒を縮めるより、不要なクエリや二重処理を減らす方が効く場合があります。

ファイル受信 → 計算 → BigQueryの改善例



対象	改善の観点
BigQueryクエリ	SELECT *を避ける。パーティションを絞る。オンデマンド課金では処理バイト数を確認
事前確認・上限	BigQueryコンソールでドライラン等の見積りと最大課金バイト数を確認。容量課金は別の評価が必要
保存・バッチ処理	1行ごとの書込を避け、許容遅延内でまとめる。バッチロード等も要件に応じて比較
通信・リージョン	接続先との配置をそろえる候補を検討。転送経路・サービス別料金も確認
ログ	ループ内の大量ログや入力全文を減らす。件数・時間・エラー分類を中心に残す

成功した処理を繰り返さない

入力ID・ファイル世代・ジョブIDなどで重複を管理します。一時的な障害だけを上限付きで再試行し、入力不備を無限に再試行しない設計にします。バッチを大きくし過ぎると、1件の失敗で多くの処理をやり直すことになるため、部分成功を保存できるかも検討します。

タイムアウトだけでは費用が止まらない

関数がタイムアウトしても、コンテナの処理や送信済みの外部ジョブが停止するとは限りません。APIの期限、キャンセル、ジョブ状態確認を設計し、再試行時に別ジョブを重複起動しないようにします。単にHTTP応答だけを先に返す設計では、業務の完了は保証できません。

巨大ファイルや長いバッチは、分割・非同期化やCloud Run jobsも候補です。1回に詰め込む量は、処理件数、失敗範囲、応答要件から決めます。

参照：[07], [09], [10], [14], [24], [25], [26], [27] / URL一覧は15–16ページ

13 計測して最適な設定を選ぶ

全処理に共通する最安設定はありません。次の手順で再現できる比較をします。

開発環境の比較用設定例（本書の提案）

```
gcloud run services update hello-function `
  --region asia-northeast1 `
  --cpu 1 --memory 512Mi --concurrency 1 `
  --min 0 --max 2 --cpu-throttling
```

業務処理の最適値ではなく、比較の基準例です。設定変更権限が必要で、リビジョン設定の変更は新しいリビジョンを作ります。再デプロイ時も採用した設定を明示して管理します。

順番	実施すること
1：入力を固定	小・通常・最大の入力を準備。結果と重複防止を検証
2：基準値を測る	初回起動と再利用時を分ける。平均・p95時間、成功率、CPU・メモリピークを記録
3：1項目ずつ変更	CPU、メモリ、同時実行数の順に候補を比較。各候補を複数回、同じ負荷で計測
4：総費用を比較	課金対象インスタンス時間・外部API費用・再試行を集計し、成功した業務1件で割る
5：採用して監視	p95・成功率・最大入力・負荷時の条件を満たす候補から費用の小さいものを採用

比較記録に残す列

リビジョン／入力サイズ／CPU／メモリ／同時実行上限／課金方式／初回・再利用／成功件数／再試行数／平均・p95秒／メモリピーク／期間総費用／成功1件の費用。p95は95%のリクエストが収まる時間です。

課金方式は実行頻度に合わせて比べる

低頻度・断続的な実行はリクエストベースを基準にします。一定の高い負荷が続く場合は、インスタンスベースも期間総額で比較します。後者は待機を含む生存期間が課金対象で、最低1分の課金時間があるため、短い単発実行で常に安いとは限りません。

Cloud Monitoringの課金対象時間とCloud Billingの実費を突き合わせます。最大インスタンス数を低くして費用だけ下げても、待機や失敗が増えれば採用しません。予算アラートは通知であり、支出の自動停止とは別の機能です。

参照：[14], [17], [18], [20], [23], [28] / URL一覧は15–16ページ

公式ドキュメント・確認先

各ページ下部の参照番号に対応します。内容は2026年10月14日に確認しました。

[01] 名称・管理方式
https://docs.cloud.google.com/run/docs/functions/comparison

[02] 関数の書き方
https://docs.cloud.google.com/run/docs/write-functions

[03] ソースからのデプロイ・必要な権限
https://docs.cloud.google.com/run/docs/deploy-functions

[04] CLIでの初回実習
https://docs.cloud.google.com/run/docs/quickstarts/functions/deploy-functions-gcloud

[05] ローカル実行
https://github.com/GoogleCloudPlatform/functions-framework-python

[06] 開発者の認証・非公開サービスの確認
https://docs.cloud.google.com/run/docs/authenticating/developers

[07] ログの確認
https://docs.cloud.google.com/run/docs/logging

[08] 実行時のサービスアカウント
https://docs.cloud.google.com/run/docs/securing/service-identity

[09] イベントトリガー
https://docs.cloud.google.com/run/docs/triggering/trigger-with-events

[10] 実装のベストプラクティス
https://docs.cloud.google.com/run/docs/tips/functions-best-practices

[11] Secret Managerとの連携
https://docs.cloud.google.com/run/docs/configuring/services/secrets

[12] リビジョンの管理
https://docs.cloud.google.com/run/docs/managing/revisions

[13] ローカルのアプリケーション認証
https://docs.cloud.google.com/docs/authentication/provide-credentials-adc

[14] 利用料金
https://cloud.google.com/run/pricing

実習・比較設定は本書の構成例です。組織ポリシーと業務要件に合わせ、公式の料金・対応設定を確認してください。

参照： / URL一覧は15–16ページ

公式ドキュメント・確認先

各ページ下部の参照番号に対応します。内容は2026年10月14日に確認しました。

[15] 対応ランタイム
https://docs.cloud.google.com/run/docs/runtimes/function-runtimes

[16] コンソールからの初回実習
https://docs.cloud.google.com/run/docs/quickstarts/functions/deploy-functions-console

[17] コスト最適化
https://docs.cloud.google.com/run/docs/tips/services-cost-optimization

[18] 課金方式
https://docs.cloud.google.com/run/docs/configuring/billing-settings

[19] CPU・起動ブースト
https://docs.cloud.google.com/run/docs/configuring/services/cpu

[20] メモリ上限
https://docs.cloud.google.com/run/docs/configuring/services/memory-limits

[21] 同時実行の考え方
https://docs.cloud.google.com/run/docs/about-concurrency

[22] 最小インスタンス
https://docs.cloud.google.com/run/docs/configuring/min-instances

[23] 設定変更コマンド
https://docs.cloud.google.com/sdk/gcloud/reference/run/services/update

[24] BigQueryクエリ最適化
https://docs.cloud.google.com/bigquery/docs/best-practices-performance-compute

[25] BigQuery費用の管理
https://docs.cloud.google.com/bigquery/docs/best-practices-costs

[26] リクエストのタイムアウト
https://docs.cloud.google.com/run/docs/configuring/request-timeout

[27] BigQueryバッチロード
https://docs.cloud.google.com/bigquery/docs/batch-loading-data

[28] Cloud Runの監視指標
https://docs.cloud.google.com/run/docs/monitoring

実習・比較設定は本書の構成例です。組織ポリシーと業務要件に合わせ、公式の料金・対応設定を確認してください。

参照： / URL一覧は15–16ページ